Code Based Learning
AMI prepared
2026.3

Databases – Hands-On SQL

SQL Practical Course

Under construction – more Units follow !

Prof. Dr. rer. nat.
Alexander Voß
FH Aachen

Unit 0x00
General Information

A few notes in advance

In this course, the text refers to attributes, columns, tables, keys, schemas or databases, just to name a few. Here is a brief definition of the terms.

- Attributes, tables: The table on the right named shop_product models products from the AMI_ZONE company. The columns of the table are the so-called attributes, i.e. the properties of each product. Each row is an entity/object - here a product.
- Standard data types in PostgreSQL: Attributes are typed, e.g.
  - integer (int): classic integer
  - numeric(p,s): exact decimal number with p digits in total and s after the point, e.g. prices
  - double precision: floating point
  - varchar(n): text with at most n characters
  - text: text of any length
  - timestamp: date and time in the form YYYY-MM-DD hh:mm:ss; date and time hold only one part
  - boolean: true or false

	name	cat…	unit
1	Spinach	1	PK
2	Four Cheese Pizza	1	PC
3	Spinach Pizza	1	PC
4	Fish Fingers	1	PK
5	Pasta Pan	1	PK
6	Carrots	2	KG
7	Onions	2	KG
8	Bananas	2	KG

Products in AMI_ZONE

A few notes in advance

- Key, id: An entity almost always has some kind of unique identifier or id, also called a key. This identifies the specific entity and is unique.
- Foreign key: An attribute in one table that represents a key in another table is called a foreign key. The exact definition and meaning follows.
- Schema/Database: Tables are grouped together in a schema. In PostgreSQL a server holds several databases, and each database several schemas. We work in the database postgres with the schemas ami_zone, ami_kemper, ...
  In other DBMS "database" and "schema" may mean the same.
- Indexes: Additional data structures, e.g. B-trees, are created automatically or intentionally to allow quick access to data.

	name	category_id
4	Fish Fingers	1
5	Pasta Pan	1
6	Carrots	2
7	Onions	2
8	Bananas	2
9	Garlic	2
10	Pork Sausage	3
11	Meatballs	3
12	Milk	4
13	Quark	4

id	name
1	Frozen Goods
2	Fruits, Vegetables
3	Sausages, Cold Meat
4	Milk Products
5	Cold Drinks
6	Barbecue Products
7	Snacks
8	Hot Drinks
9	Convenience Foods
10	Baked Goods

Products and Categories

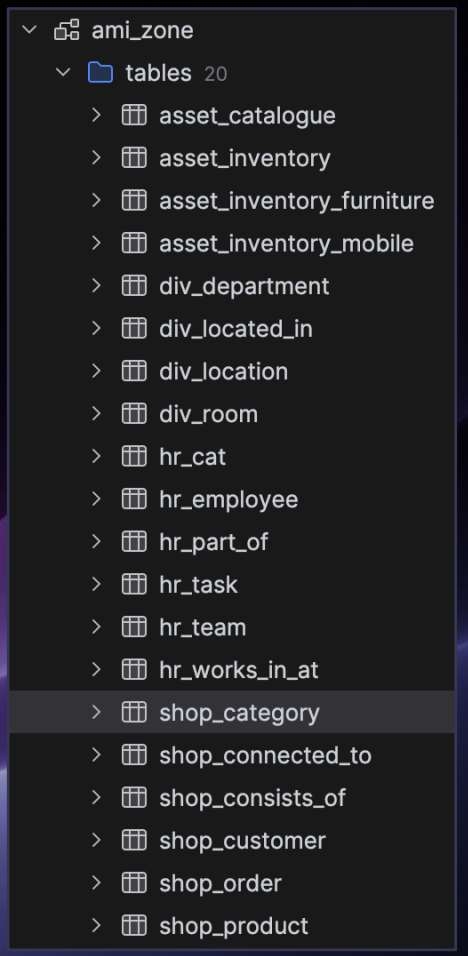
A few notes in advance

- Upper/lower case: In PostgreSQL, keywords are case-insensitive (SELECT = select). Unquoted names are folded to lower case (Product → product); names in double quotes keep their case. String comparisons are case-sensitive: 'Pizza' ≠ 'pizza'. It is not relevant to the exams.
- NULL: In addition to the typical range of values for each data type, there is another value that an attribute can have: NULL. The meaning of the value NULL is given by the context, often the value is simply used for "no specification" or "not yet specified".
  The column definition of a table may allow or disallow NULL, but if the value is allowed, it often requires special handling in queries.
- Script: The SQL commands for a unit are also available as an sql file, so you can try them out instead of typing them in.

Sample Data AMI_ZONE

The Idea

- AMI_ZONE is a fictitious company with individual departments and employees at different locations that delivers everyday necessities when there is no time to go shopping.
  Typical examples are frozen pizzas or the latest c't magazine.
- The database contains a lot of data and relationships and is used to practise SQL commands.



ami_zone database

Unit 0x01
Selection & Projection

Select & Search_Path

Intention

Query data.

Projection and Selection

- *Projection* selects columns by specifying table attributes.
- *Selection* filters rows by specifying a criterion that must be met for each row.

SEARCH_PATH (default schema)

- If a default schema or a way to find a table is specified with SEARCH_PATH the explicit specification of the schema can be omitted for tables in SQL expressions.

Examples

```sql
-- all data
SELECT * FROM ami_zone.shop_product;

-- default schema, i.e. unqualified table names refer to ami_zone
SET SEARCH_PATH = ami_zone;

-- projection example
SELECT name, price FROM shop_product;

-- selection example
SELECT * FROM shop_product WHERE price>2;
```

	name	price
1	Spinach	1.99
2	Four Cheese Pizza	2.39
3	Spinach Pizza	2.29
4	Fish Fingers	1.99
5	Pasta Pan	3.29
6	Carrots	0.39

Projection

Select * & Table Alias

Intention

Query all records and attributes of a table, and use a table alias.

Select All

- * selects all attributes in a table.
- The column headers are the attribute names or columns of the table.
- The order of the records is not specified here.

Table Alias

- Tables can be given an alias name, here P, which is used instead.
- This not only makes more complex expressions shorter, but also removes any ambiguity.
- Attributes in the query then may get or need this prefix.

Examples

```sql
-- tables with alias
SELECT P.* FROM shop_product P;

-- attributes with alias
SELECT P.name, P.price FROM shop_product P WHERE price>2;
```

name	price
Four Cheese Pizza	2.39
Spinach Pizza	2.29
Pasta Pan	3.29
Meatballs	2.35
Grill Sausages	3.55

Selection and Projection

Projection & Column Alias

Intention

Query only certain attributes (projection).

Column names

- Column names, e.g. ID, can be specified explicitly.
- AS is optional.

Quotes in PostgreSQL

- single quotes are string literals ('Pizza');
- double quotes are names, which keep their case and may contain blanks ("Price [€]").
- Unquoted names are folded to lower case: name Product gives the column product.

Example

```
-- note column names (aliases): unquoted names are folded to lower case
-- (Product -> product),
-- double quotes keep case and allow blanks; single quotes are string literals,
-- not names
SELECT id as "IDX",
  name Product,
  price "Price [€]"
FROM shop_product;
```

	IDX	product	Price [€]
1	1	Spinach	1.99
2	2	Four Cheese Pizza	2.39
3	3	Spinach Pizza	2.29
4	4	Fish Fingers	1.99
5	5	Pasta Pan	3.29
6	6	Carrots	0.39

Column Alias

Functions

Intention

Use arithmetic expressions and functions (e.g. round, concat).

Documentation

- Consulting the documentation is the best way to find out about the calling specification and possible variations of similar functions.

Example

```sql
-- calculated values
SELECT
    id "ID ",
    concat(name,' [',unit,']') "Product",
    price "Price [€]",
    round(price/(1.0+vat),2) "excl. VAT",
    round(price/(1.0+vat)*vat,2) "VAT "
FROM shop_product;
```

"ID "	"Product"	"Price [€]"	"excl. VAT"	"VAT "
1	Spinach [PK]	1.99	1.86	0.13
2	Four Cheese Pizza [PC]	2.39	2.23	0.16
3	Spinach Pizza [PC]	2.29	2.14	0.15

Functions

Operators & NULL

Intention

Use expressions such as is NULL and functions with NULL.

NULL

- Caution: Expressions are NULL if one of the operands is NULL.
- is NULL returns a boolean (true/false).
- coalesce returns the first non-NULL parameter.

Examples

```
-- calculated values involving NULL
SELECT
    brand,
    risk_of_loss_percent,
    risk_of_loss_percent is NULL "NULL?",
   coalesce(100-risk_of_loss_percent,100.00) "Fulfilment"
FROM shop_customer;
```

brand	risk_of_loss_percent	"NULL?"	"Fulfilment"
Deutsche Bank	<null>	• true	100
Börse	<null>	• true	100
E.ON	<null>	• true	100
Infineon	<null>	• true	100
Lufthansa	<null>	• true	100
RWE	2.00	false	98

Expressions with NULL

Distinct

Intention

Duplicate data should not be included.

Remarks

- Without distinct results may occur more than once. This is where the analogy with a mathematical set breaks down.
- Further attributes, e.g. name, usually do not make sense in the second query - why?

Examples

```
-- distinct values
SELECT category_id, name FROM shop_product;
SELECT distinct category_id FROM shop_product;
```

category_id	name
1	Spinach
1	Four Cheese Pizza
1	Spinach Pizza
1	Fish Fingers
1	Pasta Pan
2	Carrots
2	Onions

category_id
11
9
3
5
4
10
6

With and without distinct

Count

Intention

Determine the number of records.

Remarks

- A * in count causes all records to be counted, whereas specifying an attribute causes only records other than NULL to be counted.
- Adding distinct considers only different values.

Examples

```
-- count number of lines, with or without NULLs
select count(*) from shop_customer;
select count(discount_percent) from shop_customer;
select count(distinct discount_percent) from shop_customer;
select count(risk_of_loss_percent) from shop_customer;
```

	count
1	12

	count
1	12

	count
1	4

	count
1	1

count

WHERE

Intention

Query only table attributes or expressions matching a condition (after where).

Remarks

- The condition, here price=1.99, tests for equality. price is of type numeric(12,2), an exact decimal type, so this is safe. With floating-point types (real, double precision) the usual caveats about rounding errors apply.

Example

```
-- selection with WHERE
SELECT name, price FROM shop_product
    WHERE price=1.99;
```

	name	price
1	Spinach	1.99
2	Fish Fingers	1.99
3	Pork Sausage	1.99
4	Chips	1.99
5	Pasta in Sauce	1.99

where

Operators

Intention

Use operators such as like, not, and in where conditions.

Like

like is used for string comparison; like is case-sensitive in PostgreSQL; ilike ignores case.

Wildcards

The % wildcard allows any number of characters (including none),

The _ wildcard allows exactly one character.

Examples

```
-- string comparison with like including wildcards % and _
SELECT name, price FROM shop_product
    WHERE name like '%Pizza';
SELECT name, price FROM shop_product
    WHERE name like '%Pizza' and
    not name like '%Spinach%';
```

like

name	price
Four Cheese Pizza	2.39
Spinach Pizza	2.29

name	price
Four Cheese Pizza	2.39

Operators

Intention

Use operators such as between...and, in in where conditions.

Range

- The boundaries at between...and are inclusive.

Examples

```
-- between and in
SELECT name, price FROM shop_product
    WHERE price between 1.00 and 2.00;
SELECT name, price FROM shop_product
    WHERE price in (0.99, 1.99, 2.99);
```

name	price
Spinach	1.99
Fish Fingers	1.99
Bananas	1.29
Pork Sausage	1.99
Cola light	1.50

name	price
Spinach	1.99
Fish Fingers	1.99
Pork Sausage	1.99
Quark	0.99
Water	0.99

between...and & in

Case

Intention

The case statement allows an output per case.

Example

```
-- case
SELECT name, price,
       case
            when price <= 2.00 then 'cheap'
            when price <= 5.00 then 'ok'
            else 'expensive'
       end as affordable
    FROM shop_product
    WHERE price between 1.00 and 10.00;
```

name	price	affordable
Spinach	1.99	cheap
Four Cheese Pizza	2.39	ok
Spinach Pizza	2.29	ok
Fish Fingers	1.99	cheap
Pasta Pan	3.29	ok

case

And & Or

Intention

Use operators such as and, or, is NULL, is not NULL and parentheses in where conditions.

NULL

- As before, be careful with NULL values.
  The result of an arithmetic operation with NULL is also NULL.

Examples

```sql
-- different clauses
SELECT name, price FROM shop_product
    WHERE (price<1.00 or price>5.00)
        and category_id=11;
SELECT brand, risk_of_loss_percent FROM shop_customer
    WHERE risk_of_loss_percent is not null;
```

and & or & is not NULL

brand	risk_of_loss_percent
RWE	2.00

name	price
Bild	0.90
GameStar	6.50

Sort by

Intention

Sort the found results by criteria (order by), ascending (asc), or descending (desc).

Asc & Desc

- In the lower result, products with the same price are additionally sorted by name, descending.

Examples

```sql
-- order by, asc, desc
SELECT name, price FROM shop_product
    WHERE category_id in (1,3,4) ORDER BY price; -- asc
SELECT name, price FROM shop_product
    WHERE category_id in (1,3,4) ORDER BY price, name desc;
```

name	price
Milk	0.79
Yoghurt	0.98
Quark	0.99
Pork Sausage	1.99
Fish Fingers	1.99
Spinach	1.99
Spinach Pizza	2.29
Meatballs	2.35
Four Cheese Pizza	2.39
Pasta Pan	3.29

name	price
Milk	0.79
Yoghurt	0.98
Quark	0.99
Spinach	1.99
Pork Sausage	1.99
Fish Fingers	1.99
Spinach Pizza	2.29
Meatballs	2.35
Four Cheese Pizza	2.39
Pasta Pan	3.29

order by

Interactive

Intention

Interactive use of select.

Remarks

- In PostgreSQL a select needs no FROM clause.
  Some DBMS, such as Oracle, require a table and provide the pseudo table dual for this; PostgreSQL has no dual.

Example

```
-- interactive usage, a SELECT does not need a FROM clause in PostgreSQL
-- (Oracle needs 'FROM dual' here, PostgreSQL does not know DUAL)
select now(), 1+2*3, random();
```

now	?column?	random
2026-10-06 20:40:12.393198 +00:00	7	0.5180233623461439

Some expression

Check

Tasks

1. Print the entire contents of each of the tables `div_department`, `shop_customer`, `hr_employee`, `shop_product`, `hr_team` and `shop_category` from `ami_zone`.
2. Find out how many different `VAT` rates there are in the `shop_product` table and print them.
3. Which customers are a *Bank* or *Sparkasse* by name? Identify them and print only the name and discount. Remember: `like` is case-sensitive, `ilike` is not.
4. Find the monthly salary (assume 12 months) of all employees whose manager is Mia or Ben. Give the column the heading `Monthly salary` and round the monthly salary to full Euros. (Look up the id of Mia and Ben, no sub-query required yet).
5. How many employee names begin with an `M`? And how many contain an `M`? Find out the number of each.

Unit 0x02
Joins

Motivation

Relationship product-category

- Each product is assigned to a category (N:1) and this relationship is realised by the foreign key category_id in shop_product.
- It would be nice if we could obtain the category itself instead of the category_id:

Examples

```
SELECT P.id,P.name,P.category_id FROM shop_product P;
SELECT C.id,C.name FROM shop_category C;

-- goal
SELECT P.id,P.name,C.name FROM shop_product P
    JOIN shop_category C  1..n<->1:  on C.id = P.category_id;
```

	name	category_id
1	Spinach	1
2	Four Cheese Pizza	1
3	Spinach Pizza	1
4	Fish Fingers	1
5	Pasta Pan	1
6	Carrots	2
7	Onions	2

Products

	name
1	Frozen Goods
2	Fruits, Vegetables
3	Sausages, Cold Meat
4	Milk Products
5	Cold Drinks
6	Barbecue Products
7	Snacks

Categories

	P.name	C.name
1	Spinach	Frozen Goods
2	Four Cheese Pizza	Frozen Goods
3	Spinach Pizza	Frozen Goods
4	Fish Fingers	Frozen Goods
5	Pasta Pan	Frozen Goods
6	Carrots	Fruits, Vegetables
7	Onions	Fruits, Vegetables

Join

Note: DataGrip sometimes shows the cardinality of the relationship (1..n<->1). This is not part of SQL. !

Motivation

Relationship product–category

- First approach: If we specify both tables after `from` in the select, we get the Cartesian product - in short, every product with every category (a so-called implicit cross join).
- How can we cleverly select so that the result contains only matching rows?
  In other words: What is the concrete `where` condition?

Example

```
-- motivation: cross
SELECT P.id,P.name,P.category_id,C.id,C.name
    FROM shop_product P, shop_category C;
```

P.id	P.name	category_…	C.id	C.name
38	Computerbild	11	1	Frozen Goods
39	Zeit	11	1	Frozen Goods
40	GameStar	11	1	Frozen Goods
1	Spinach	1	2	Fruits, Vegetables
2	Four Cheese Pizza	1	2	Fruits, Vegetables
3	Spinach Pizza	1	2	Fruits, Vegetables

Cross join

Motivation

Relationship product–category

- We are only interested in the rows where the foreign key category_id matches the category id - this obviously results in the correct pairing and is explicitly expressed by the where condition.
- The DBMS recognises this join and does not first determine all the combinations, only to filter out almost all of them again…

Example

```
-- motivation: explicit join
SELECT P.id,P.name,P.category_id,C.id,C.name
    FROM shop_product P, shop_category C
    WHERE P.category_id=C.id;
```

P.id	P.name	category_…	C.id	C.name
1	Spinach	1	1	Frozen Goods
2	Four Cheese Pizza	1	1	Frozen Goods
3	Spinach Pizza	1	1	Frozen Goods
4	Fish Fingers	1	1	Frozen Goods
5	Pasta Pan	1	1	Frozen Goods
6	Carrots	2	2	Fruits, Vegetables
7	Onions	2	2	Fruits, Vegetables

Join with where

Motivation

Relationship product-category

- This join variant uses an explicit join command and the on condition is identical to the where condition before.

Example

```
-- motivation: join by command
SELECT P.id,P.name,C.name
    FROM shop_product P
    JOIN shop_category C  1..n<->1: ON P.category_id=C.id;
```

	P.name	C.name
1	Spinach	Frozen Goods
2	Four Cheese Pizza	Frozen Goods
3	Spinach Pizza	Frozen Goods
4	Fish Fingers	Frozen Goods
5	Pasta Pan	Frozen Goods
6	Carrots	Fruits, Vegetables
7	Onions	Fruits, Vegetables

Inner join

Cross Join

Intention

- Specifying multiple tables with no further restrictions results in the cross product of the entries - an implicit *cross join* (first select). That is, each entity in the first table is combined with each entity in the second table (similarly for multiple tables).
- An explicit *cross join* (second select) is equivalent.

Examples

```
-- cross join implicitly and explicitly
SELECT P.id, P.name, P.category_id, C.id, C.name
    FROM shop_product P,shop_category C;

SELECT P.id, P.name, P.category_id, C.id, C.name
    FROM shop_product P CROSS JOIN shop_category C;
```

P.id	P.name	category_…	C.id	C.name
1	Spinach	1	1	Frozen Goods
2	Four Cheese Pizza	1	1	Frozen Goods
3	Spinach Pizza	1	1	Frozen Goods
4	Fish Fingers	1	1	Frozen Goods
5	Pasta Pan	1	1	Frozen Goods
6	Carrots	2	1	Frozen Goods
7	Onions	2	1	Frozen Goods

cross join

Inner Join

Intention

- As described above, you are often only interested in data that 'belongs together', such as the category of a product.
- An *inner join* is defined as a selection on the cross-product. Both joins are equivalent here, but the upper form is considered obsolete.

Remarks

- The explicit inner join expects an `on` specifying which data belongs together. `inner` is optional here.

Examples

```
-- inner join (old, but this is the formal definition)
SELECT P.id, P.name, P.category_id, C.id, C.name
    FROM shop_product P,shop_category C
    WHERE P.category_id=C.id;
-- inner join, use this
SELECT P.id, P.name, P.category_id, C.id, C.name
    FROM shop_product P INNER JOIN shop_category C
    1..n<->1: ON P.category_id=C.id;
```

P.id	P.name	category_…	C.id	C.name
1	Spinach	1	1	Frozen Goods
2	Four Cheese Pizza	1	1	Frozen Goods
3	Spinach Pizza	1	1	Frozen Goods
4	Fish Fingers	1	1	Frozen Goods
5	Pasta Pan	1	1	Frozen Goods
6	Carrots	2	2	Fruits, Vegetables
7	Onions	2	2	Fruits, Vegetables

inner join

Inner Join and Self Join

Example

- All customers who have a purchase order - and only them!
- All employees together with the manager.

Remarks

- The latter is a self-relationship, a so-called *self-join*.

Example 1 & 2

```
-- example 1, inner join
SELECT O.id, O.customer_id, C.id, C.brand
    FROM shop_order O
    INNER JOIN shop_customer C  1..n<->1: ON O.customer_id=C.id;

-- example 2, self-join
SELECT E.id,E.name,E.employee_id,EE.id,EE.name "Boss"
    FROM hr_employee E
    INNER JOIN hr_employee EE on E.employee_id=EE.id;
```

O.id	customer…	C.id	brand
1	12	12	Sparkasse
2	12	12	Sparkasse
3	7	7	E.ON

E.id	name	employee_…	EE.id	"Boss"
1	Board	1	1	Board
2	Paul	1	1	Board
3	Hannah	1	1	Board
4	Luka	1	1	Board
5	Mia	1	1	Board
6	Ben	5	5	Mia
7	Emma	5	5	Mia

Self-join

Inner Join, Multiple Tables

Example 3

Example

- Departments can be spread across multiple locations and locations can host multiple departments.
- This is an N:M relation.

Remarks

- Multiple joins can be easily combined. The definition remains the same.

```
SELECT LI.department_id, D.name, LI.location_id, L.place
    FROM div_located_in LI
    JOIN div_department D  1..n<->1: ON LI.department_id=D.id
    JOIN div_location L  1..n<->1: ON LI.location_id=L.id;
```

department_id	name	location…	place
1	Board	4	Berlin
2	Human Resources	4	Berlin
3	Marketing and Sales	1	Aachen
3	Marketing and Sales	2	Jülich
3	Marketing and Sales	3	Köln
3	Marketing and Sales	4	Berlin
4	Finance	4	Berlin

Multiple joins

Inner Join

Example 4

Example

- Here you can see who works in which department, what they work on and how many hours they work each week.

Remarks

- Although this is a relational table, the entities have an attribute, the hours. They depend on the employee, the department and also on the task.

```
SELECT E.name,D.name,T.name,W.hours_per_week
    FROM hr_works_in_at W
    JOIN hr_employee E  1..n<->1: on E.id = W.employee_id
    JOIN div_department D  1..n<->1: on D.id = W.department_id
    JOIN hr_task T  1..n<->1: on T.id = W.task_id;
```

E.name	D.name	T.name	hours_per_week
Board	Board	Regular tasks	0.00
Paul	Board	Regular tasks	5.00
Hannah	Board	Regular tasks	5.00
Luka	Board	Regular tasks	5.00
Mia	Board	Regular tasks	30.00
Mia	Finance	Project XCoin	10.00
Ben	Research and Development	Regular tasks	20.00
Ben	Project Management	Regular tasks	20.00
Emma	Human Resources	Regular tasks	20.00

Multiple joins

Inner Join, Missing Counterpart

Example

- Here, all cats are associated with the people who care for them. hr_cat has three cats; poor Roy has no person (NULL).

Remarks

- Note that the entities for which there is no counterpart (i.e. containing a NULL in the foreign key) are dropped (here Roy). This is a fundamental property of the inner join.

Example 5

```
-- example 5
SELECT C.* FROM hr_cat C;
SELECT C.id,C.name, E.id, E.name
    FROM hr_cat C JOIN hr_employee E
    1..n<->1: ON E.id = C.employee_id;
```

C.id	C.name	E.id	E.name
1	Mauzi	2	Paul
2	Mini	17	Max

Inner join with NULL

Outer Join

Intention

- Query 'related' data and data for which there is no counterpart.
- Because there are two sides to the relationship, you also need to specify in the outer join which side you want to have completely in the result set.
  There are left outer join, right outer join and full outer join:
  - A left outer join considers all entities in the entity set on the left of the relationship, even if there are no related entities in the entity set on the right. These attributes are then `NULL`.
  - The right outer join is similar, but with the roles reversed. This means that it considers all entities on the right side, and fills in the attributes of the 'related' entity on the left, or `NULL`.
  - The full outer join is the union of the left outer join and the right outer join – PostgreSQL supports `FULL OUTER JOIN` directly.

Outer Join

Example

- For the *left outer join*, Roy is included because all entities of hr_cat, left of the join, are considered, even if they are not in relation.
- The associated attributes of the non-existent partners are NULL.
- If we look at the same SQL command as a *right outer join*, all entities from hr_employee, right side of the join, appear and the associated attributes are NULL, if the employee is not related to a cat. Hence Roy is not included.
- Please note that it is the side in the join that is important, not which of the attributes are output first.

Examples

```
-- outer join (left) vs inner join
SELECT C.id,C.name, E.id, E.name
    FROM hr_cat C LEFT OUTER JOIN hr_employee E
    1..n<->0..1: ON E.id = C.employee_id;

SELECT C.id,C.name, E.id, E.name
    FROM hr_cat C RIGHT OUTER JOIN hr_employee E
    0..n<->1: ON E.id = C.employee_id;
```

C.id	C.name	E.id	E.name
1	Mauzi	2	Paul
2	Mini	17	Max
3	Roy	<null>	<null>

C.id	C.name	E.id	E.name
1	Mauzi	2	Paul
2	Mini	17	Max
<null>	<null>	20	Leonie

Left and right outer join

Full Outer Join

Example

- The full outer join is the union of the left outer join and the right outer join. This means that all entities on both sides are included, with NULL entries only if necessary.
- PostgreSQL has FULL OUTER JOIN. Some DBMS (MySQL/MariaDB) do not; there the left and the right outer join are combined with union.

Remarks

- Switching the tables in the join and the type of left or right outer join, results in the original outer join.
- Because inner joins only affect entities that have a partner, there is no left or right inner join.

Examples

```
-- outer join (full)
SELECT C.id,C.name, E.id, E.name
    FROM hr_cat C FULL OUTER JOIN hr_employee E
    0..n<->0..1: ON E.id = C.employee_id;
```

```
SELECT C.id,C.name, E.id, E.name
    FROM hr_cat C LEFT OUTER JOIN hr_employee E
    1..n<->0..1: ON E.id = C.employee_id
union
SELECT  id C.id,  name C.name,  id E.id,  name E.name
    FROM hr_cat C RIGHT OUTER JOIN hr_employee E
    0..n<->1: ON E.id = C.employee_id;
```

C.id	C.name	E.id	E.name
1	Mauzi	2	Paul
2	Mini	17	Max
3	Roy	<null>	<null>
<null>	<null>	20	Leonie
<null>	<null>	11	Finn
<null>	<null>	12	Lea

Full outer join

Inner or Outer Join

Remarks

- Whether it is an inner or outer join can often be seen by looking at how it is worded.

Example

- Only those customers for whom an order exists, ...
  i.e. those who are in a relationship - thus inner join.
- All customers for whom an order has been placed and those who have not yet placed an order,...
  i.e. all customers, with or without partners,
  i.e. outer join.

Examples

```
SELECT O.id, O.customer_id, C.id, C.brand
    FROM shop_order O INNER JOIN shop_customer C
    1..n<->1: ON O.customer_id= C.id;

SELECT O.id, O.customer_id, C.id, C.brand
    FROM shop_order O RIGHT OUTER JOIN shop_customer C
    0..n<->1: ON O.customer_id=C.id;
```

.id	customer…	C.id	brand
1	12	12	Sparkasse
2	12	12	Sparkasse
3	7	7	E.ON

O.id	customer…	C.id	brand
1	12	12	Sparkasse
2	12	12	Sparkasse
3	7	7	E.ON
<null>	<null>	11	SAP
<null>	<null>	10	RWE

Inner and outer join

Natural Join

Intention

- For tables with attributes with the same name and the same meaning (!), the explicit specification of the condition (with `on`, `using` or `where`) for an inner join can be omitted. The DBMS then uses all attributes with the same name for a so-called *natural join*.

Remarks

- The DBMS always uses all attributes with the same name, i.e. adding new attributes or renaming existing ones can change the query!
- Note that extending a table is nothing unusual and can even happen automatically, for example when the object structure to be persisted is extended and the tables reflect the classes. This happens quickly, without taking into account possible natural joins - and then the queries do not return correct results (but technically still work).

As a rule of thumb, the use of natural joins is not recommended! !

Natural Join

Remarks

- In the ami_zone database, natural joins are not suitable because, on the one hand, all primary keys are called id and all foreign keys <table>_id, i.e. these relationships must be specified explicitly, and on the other hand, all attributes with the same name, e.g. name, have different meanings, so they are also excluded.

Example

- In the example database ami_kemper, lectures are identified by a VorlNr and students by a MatrNr. The hoeren relationship relates MatrNr to VorlNr and a natural join can be used there.

Examples

```
SELECT S.Name,V.Titel
    FROM Studenten S
    NATURAL JOIN hoeren R
    NATURAL JOIN Vorlesungen V;

SELECT S.Name,V.Titel
    FROM Studenten S
    JOIN hoeren R  1<->1..n: ON S.MatrNr = R.MatrNr
    JOIN Vorlesungen V  1..n<->1: ON R.VorlNr = V.VorlNr;
```

	name	titel
1	Jonas	Glaube und Wissen
2	Fichte	Grundzuege
3	Schopenhauer	Logik
4	Schopenhauer	Grundzuege
5	Carnap	Ethik
6	Carnap	Wissenschaftstheorie

	name	titel
1	Jonas	Glaube und Wissen
2	Fichte	Grundzuege
3	Schopenhauer	Logik
4	Schopenhauer	Grundzuege
5	Carnap	Ethik
6	Carnap	Wissenschaftstheorie

Natural joins

Check

Tasks

1. The required tables shop_order and shop_customer are taken from the ami_zone schema.
   Query the delivery time and customer name for all orders:
   (a) Use an inner join with where once,
   (b) and an inner join with (inner) join.
   (c) Now add the name of the agent from hr_employee.

Results

"Delivery"	"Customer"
2025-10-13 13:14:00.000000	Sparkasse
2025-10-13 17:38:00.000000	Sparkasse
2025-10-14 08:22:00.000000	E.ON

"Delivery"	"Customer"	"Agent"
2025-10-13 13:14:00.000000	Sparkasse	Anna
2025-10-13 17:38:00.000000	Sparkasse	Finn
2025-10-14 08:22:00.000000	E.ON	Anna

Check

Tasks

2. For each department (div_department), output the location (div_location). They are related by div_located_in.
   (a) Use an (inner) join with join.
   (b) Also output departments that do not (yet) have a location.
   (c) Formulate (b) once as a left and once as a right outer join.
   (d) Use an SQL command to answer the question of which departments do not yet have a location.

Results

	name	place
1	Board	Berlin
2	Human Resources	Berlin
3	Marketing and Sales	Aachen
4	Marketing and Sales	Jülich
5	Marketing and Sales	Köln
6	Marketing and Sales	Berlin
7	Finance	Berlin
8	Research and Development	Aachen
9	Research and Development	Jülich
10	Project Management	Köln
11	Project Management	Berlin

	name	place
1	Board	Berlin
2	Human Resources	Berlin
3	Marketing and Sales	Aachen
4	Marketing and Sales	Jülich
5	Marketing and Sales	Köln
6	Marketing and Sales	Berlin
7	Finance	Berlin
8	Research and Development	Aachen
9	Research and Development	Jülich
10	Project Management	Köln
11	Project Management	Berlin
12	Vehicle Fleet	<null>

	name	place
1	Board	Berlin
2	Human Resources	Berlin
3	Marketing and Sales	Aachen
4	Marketing and Sales	Jülich
5	Marketing and Sales	Köln
6	Marketing and Sales	Berlin
7	Finance	Berlin
8	Research and Development	Aachen
9	Research and Development	Jülich
10	Project Management	Köln
11	Project Management	Berlin
12	Vehicle Fleet	<null>

name
Vehicle Fleet

Check

Tasks

3. Now we are talking about employees (hr_employee), managers (boss) (foreign key employee_id in hr_employee), departments (div_department) and tasks (hr_task), related by hr_works_in_at.
   (a) Output all employees whose manager is Mia.
   (b) Print all employees with their name, department, task, hours per week and manager's name.
   Tip: Start with the relationship hr_works_in_at.

(Partial) Results

name	"Manager"
Ben	Mia
Emma	Mia
Sofia	Mia
Jonas	Mia

E1.name	D.name	T.name	hours_per_week	"Manager"
Board	Board	Regular tasks	0.00	Board
Paul	Board	Regular tasks	5.00	Board
Hannah	Board	Regular tasks	5.00	Board
Luka	Board	Regular tasks	5.00	Board
Mia	Board	Regular tasks	30.00	Board
Mia	Finance	Project XCoin	10.00	Board
Ben	Research and Development	Regular tasks	20.00	Mia
Ben	Project Management	Regular tasks	20.00	Mia

Check

Tasks

4. Analyse a customer network (shop_customer and shop_connected_to). Note that the knowledge 'customer a' knows 'customer b' is neither commutative nor transitive for this task.
   (a) Output the names of all connections of banks and savings banks (customer a has 'bank' or 'kasse' in its name regardless of upper/lower case) in the customer network.
   (b) Output all customers (customer a) that have not yet established connections.

Results

C1.brand	C2.brand
Sparkasse	Börse
Deutsche Bank	Sparkasse
Commerzbank	Deutsche Bank
Commerzbank	Sparkasse

brand
Infineon
Börse

Unit 0x03
Group Functions

Goal

Goal

Grouping data according to specific criteria and aggregating data are important functions. These are commonly known as Group Functions or Aggregation Functions.

Group Functions

Intention

Grouping data and using group functions.

Example

- Here, we calculate the listed aggregation functions for all products. These include Minimum, Maximum, Sum, Count, Average, and others.
- stddev_pop refers to the so-called 'population standard deviation', in contrast to stddev_samp 'sample standard deviation'.

Example

```
SELECT min(price), max(price), sum(price), count(price),
       sum(price)/count(price), avg(price), stddev_pop(price),
       sqrt(var_pop(price)) FROM shop_product;
```

min	max	sum	count	?colum…	avg	stddev_pop	sqrt
0.29	6.5	85.55	40	2.13875	2.13875	1.125475871576108	1.125475871576108

Group functions min, max, count, …

Count

Intention

- Use Group Functions on selections.

Examples

- count(*) counts all lines including NULL (does not occur here)
- count(price) counts without NULL
- count(distinct price) counts only different prices

Remarks

- Consider whether to include NULL values or duplicate values.

Examples

```
SELECT count(*), avg(price) FROM shop_product WHERE category_id=1;
SELECT count(price), avg(price) FROM shop_product WHERE category_id=1;
SELECT count(distinct price), avg(distinct price) FROM shop_product WHERE category_id=1;
```

count	avg
5	2.39

count	avg
5	2.39

count	avg
4	2.49

count

GROUP BY

Intention

- Use Group Functions on grouped data.

Examples

- group by divides the set of rows into groups; the group functions are calculated per group
- Refer to the previous example for count, min, max and avg.
- Without order by, the order of the groups is undefined.

Example

```
-- Note: without ORDER BY the order of the groups is undefined.
SELECT count(price),category_id,min(price),max(price),avg(price) FROM shop_product
GROUP BY category_id ORDER BY category_id;
```

count	catego…	min	max	avg
5	1	1.99	3.29	2.39
4	2	0.29	1.29	0.7375
2	3	1.99	2.35	2.17
3	4	0.79	0.99	0.92
4	5	0.99	1.8	1.4475
3	6	2.79	3.55	3.2766666666666667
3	7	1.59	1.99	1.79

group by

GROUP BY

Intention

- Use Group Functions on grouped data of a selection.

Examples

- As before, but the set of rows is first filtered to category 1, 2 or 4.

Example

```
-- Use Group Functions on grouped data of a selection.
SELECT count(price),category_id,min(price),max(price),avg(price) FROM shop_product
WHERE category_id IN (1,2,4) GROUP BY category_id ORDER BY category_id;
```

count	catego…	min	max	avg
5	1	1.99	3.29	2.39
4	2	0.29	1.29	0.7375
3	4	0.79	0.99	0.92

where & group by

GROUP BY & HAVING

Intention

- Use Group Functions on grouped data with condition.

Examples

- Condition having filters the subsets grouped with group by.
- This time, we are applying the condition to the outcome of the grouping process.
- Please note that there is often confusion over whether to use where or having.

Example

```
-- Use Group Functions on grouped data with condition.
SELECT count(price),category_id,min(price),max(price),avg(price) FROM shop_product
WHERE category_id IN (1,2,4) GROUP BY category_id HAVING min(price)>1
ORDER BY category_id;
```

count	catego…	min	max	avg
5	1	1.99	3.29	2.39

having

Correct Grouping

Intention

- Use Group Functions correctly.

Remark

- If you group products by their category, the name and the unit of each product may differ. While it is technically feasible to include them in the select statement, it is nonsensical: PostgreSQL rejects it with an error. MySQL/MariaDB (without ONLY_FULL_GROUP_BY) silently return an arbitrary row per group – which is worse.
- Only use attributes that you know to be the same within the group. In some cases, you may need to limit the attributes to those specified in the group by condition.

Examples

```
SELECT name, category_id, price, unit FROM shop_product
GROUP BY category_id;
```

```
[42803] ERROR: column "shop_product.name" must appear in the GROUP BY clause or be used in an aggregate function
  Position: 8
```

Wrong attribute usage

HAVING WITH ALIAS

Intention

- Use Group Functions with alias.

Examples

- The lowest value gets the alias S – shown as s, because unquoted names are folded to lower case.
- S can be used in order by, but not in where, group by or having, because these clauses are evaluated before select.
- In having, repeat the expression: 3*min(price)>1.

Examples

```
-- Use Group Functions with alias.
-- An alias (here S) may be used in ORDER BY, but not in WHERE, GROUP BY or HAVING,
-- because these clauses are evaluated before SELECT; repeat the expression instead.
SELECT count(price),category_id,min(price) S FROM shop_product
WHERE category_id IN (1,2,4)
GROUP BY category_id HAVING 3*min(price)>1
ORDER BY S;
```

cou…	catego…	s
3	4	0.79
5	1	1.99

With alias

Multi-level Grouping

Intention

- Use group by with multiple attributes.

Examples

- Products are grouped by VAT and then by unit.

Examples

```
-- Use group by with multiple attributes.
SELECT count(*) FROM shop_product WHERE unit='KG' AND VAT=0.07;  --  4
SELECT count(*) FROM shop_product WHERE unit='PK' AND VAT=0.07;  -- 14
SELECT count(*) FROM shop_product WHERE unit='PC' AND VAT=0.07;  -- 16

SELECT count(id) FROM shop_product WHERE unit='PC' AND VAT=0.19; --  6

SELECT count(VAT),VAT,count(unit),unit,min(price),max(price)
FROM shop_product GROUP BY VAT, unit ORDER BY VAT, unit;
```

cou…	vat	count	unit	min	max
4	0.07	4	KG	0.29	1.29
16	0.07	16	PC	0.79	6.5
14	0.07	14	PK	1.59	3.55
6	0.19	6	PC	0.99	3.2

Multiple attributes

With Joins

Intention

- Use Group Functions with Joins.

Examples

- The top table displays a summary of all drinks. The second table groups them by product category.
- C.name must be in group by as well – PostgreSQL insists, even though the name is the same within a category.
- like is case-sensitive, hence ilike.

Examples

```
SELECT P.category_id, P.price, C.name FROM shop_product P
INNER JOIN shop_category C 1..n<->1: ON P.category_id= C.id
WHERE C.name ILIKE '%drinks';

-- C.name must be grouped as well (P.category_id alone is not enough for PostgreSQL)
SELECT count(P.price),P.category_id,avg(P.price),C.name FROM shop_product P
INNER JOIN shop_category C 1..n<->1: ON P.category_id=C.id
WHERE C.name ILIKE '%drinks' GROUP BY P.category_id, C.name
ORDER BY P.category_id;
```

catego…	price	name
5	1.50	Cold Drinks
5	1.50	Cold Drinks
5	1.80	Cold Drinks
5	0.99	Cold Drinks
8	2.50	Hot Drinks
8	3.20	Hot Drinks

count	catego…	avg	name
4	5	1.4475	Cold Drinks
2	8	2.85	Hot Drinks

With joins

Overview

- where selects the set of rows that are grouped, while
- group by groups and applies aggregation functions as needed. Note that these cannot be used in the where condition.
- having is used to condition the results of grouping.
- Selected attributes must be in the group by or inside a group function.
- Aliases can be used in order by; in having, repeat the expression.
- Joins can also be included in the grouping.

Check

Tasks

1. The required tables are taken from ami_zone. Firstly, examine the employees' wage structure, who earn more than zero annually, and arrange them according to their manager's id.
   (a) Then, group the actual yearly pays according to the supervisor's id, find the lowest and the highest pay in each group, count the number of salaries in each group and sort them. Verify the validity of the outcome.
   (b) Finally, include the average salary in the output and only produce entries where it exceeds 50,000.

Results

name	count	min	max
Board	4	5000	110000
Mia	4	50000	90000
Ben	4	50000	70000
Emma	1	70000	70000
Sofia	3	24000	60000
Lea	3	12000	50000
Leon	3	18000	50000

name	count	min	max	avg
Mia	4	50000	90000	80000
Ben	4	50000	70000	55000
Emma	1	70000	70000	70000

Check

Tasks

2. Questions regarding the product range.
   (a) What is the average price of a pizza (hint: ilike) and how many pizzas are available?
   (b) How many products are in category 1 and 2? Also print the category name.
   (c) Calculate the number of products per category with a minimum unit price of over €2.
   (d) How many items are there priced between €1 and €2 per sales tax rate?

pizza	avg
2	2.34

name	count
Frozen Goods	5
Fruits, Vegetables	4

name	count	min
Barbecue Products	3	2.79
Hot Drinks	2	2.5

count	vat
12	0.07
3	0.19

Results

Check

Tasks

3. Firstly, examine the hr_works_in_at relationship in the DBMS. An employee may work in multiple departments in varying roles.
   Who are the employees working on more than one task within a single department?

   Advice: Start with simple queries and groupings, perhaps without joins, and gradually increase the complexity.
   Check the chosen examples to ensure plausibility.

Results

A.name	M.name	tasks
Research and Development	Max	2
Research and Development	Leonie	2
Project Management	Leon	2
Research and Development	Finn	2
Finance	Emma	2